第 9 讲：堆与优先队列——反复取出当前最重要的元素

0. 本讲的核心问题

假设数据不断到来，而你需要反复回答：

- 当前最小的任务是什么？
- 当前最大的 `k` 个数是什么？
- 第 `k` 大的元素是什么？
- 多个候选状态中，下一步应优先处理哪一个？

每次都把全部数据重新排序当然可以，但通常太浪费。

堆解决的问题是：

> 不要求所有元素完全有序，只保证最重要的元素随时位于顶部。

Python 的 `heapq` 默认实现最小堆，因此堆顶 `heap[0]` 始终是当前最小元素。

---

1. 优先队列与普通队列的区别

普通队列按照进入顺序处理：

优先队列按照优先级处理：

例如医院分诊不会单纯按到达时间处理，而会优先处理情况更紧急的患者。

优先队列是一种抽象接口：

```
push(item)
peek()
pop()
```

堆是实现优先队列的常用数据结构。

---

2. 堆的结构

最小堆满足：

> 每个父节点的值都不大于它的子节点。

例如：



```
          9   8  7
```

父节点不一定比整棵子树中的所有元素都先经过排序比较，但由于堆性质沿路径成立，根节点一定是全局最小值。

注意：

```
[2, 5, 4, 9, 8, 7]
```

是一个合法最小堆，但它不是一个完全排序的数组。`5` 可以出现在 `4` 前面。

2.1 为什么堆可以存在 Python list 中

堆是一棵完全二叉树：除了最后一层以外每层都填满，最后一层从左向右填充。

因此不需要为每个节点保存 `left` 和 `right` 引用，可以按层放入数组：

```
 0  1  2  3  4  5
[2, 5, 4, 9, 8, 7]
```

对下标 `i`：

```
[python]
left = 2 * i + 1
right = 2 * i + 2
parent = (i - 1) // 2
```

在 LeetCode 中通常不需要自己实现上浮和下沉，但理解数组与树的对应关系有助于理解复杂度。

---

3. 插入和删除为什么是 `O(log n)`

3.1 插入：先放到末尾，再向上调整

把 `1` 加入下面的堆：

```
[2, 5, 4, 9, 8, 7]
[2, 5, 4, 9, 8, 7, 1]
```

新元素可能小于父节点，因此不断与父节点交换，直到恢复堆性质。这叫上浮。

完全二叉树的高度是 `O(log n)`，所以最多交换 `O(log n)` 次。

3.2 删除堆顶：末尾补到根，再向下调整

删除根节点后，为了保持完全二叉树形状：

1. 把最后一个元素移到根；
2. 比较它与子节点；
3. 与更小的子节点交换；
4. 继续向下，直到恢复堆性质。

这叫下沉，同样最多经过树高 `O(log n)`。

3.3 常见复杂度

操作	时间复杂度
查看堆顶 `heap[0]`	`O(1)`
插入 `heappush`	`O(log n)`
弹出堆顶 `heappop`	`O(log n)`
把列表原地建堆 `heapify`	`O(n)`
在堆中查找任意值	`O(n)`

堆只对“最优元素”提供快速访问，并不擅长查找任意元素。

---

4. Python `heapq` 的基本使用

```
[python]
import heapq

heap = []

heapq.heappush(heap, 5)
heapq.heappush(heap, 2)
heapq.heappush(heap, 8)

smallest = heap[0]          # 2
x = heapq.heappop(heap)     # 2
```

不要通过打印结果判断堆是否正确：

```
[python]
print(heap)
```

输出只保证满足堆性质，不保证从小到大排列。

如果需要把所有元素从小到大取出：

```
[python]
result = []

while heap:
    result.append(heapq.heappop(heap))
```

4.1 对已有列表建堆

```
[python]
nums = [7, 2, 9, 1, 5]
heapq.heapify(nums)
```

`heapify` 会原地改变 `nums`，时间为 `O(n)`。

如果不能修改原列表：

```
[python]
heap = nums.copy()
heapq.heapify(heap)
```

---

5. Python 怎样实现最大堆

`heapq` 传统接口是最小堆。为了优先取出最大值，可以存相反数：

```
[python]
```

```
import heapq

max_heap = []

for x in [4, 9, 2]:
    heapq.heappush(max_heap, -x)

largest = -heapq.heappop(max_heap)  # 9
```

真实数值越大，相反数越小，因此会先从最小堆弹出。

常见错误是只在压入时取负，弹出后忘记恢复：

```
[python]
x = -heapq.heappop(max_heap)
```

变量名最好直接写成 `max_heap`，提醒自己其中存的是相反数。

---

6. 堆中存元组：优先级与数据

实际题目往往不只保存一个数字，而要同时保存优先级和状态：

```
[python]
heapq.heappush(heap, (priority, item))
```

Python 比较元组时先比较第一个元素；相等时再比较第二个：

```
(2, "B") < (5, "A")
```

因此可以写：

```
[python]
tasks = []
heapq.heappush(tasks, (3, "   "))
heapq.heappush(tasks, (1, "    "))

priority, task = heapq.heappop(tasks)
```

6.1 优先级相同时的比较问题

如果第二项是不可比较的自定义对象，优先级相同时可能报错。可以加入唯一序号：

```
[python]
heapq.heappush(heap, (priority, sequence, item))
```

`sequence` 可以使用递增整数，保证前两项已经能确定顺序，不需要比较 `item`。

---

7. 核心模式：维护大小为 `k` 的堆

题目：在大量数字中找最大的 `k` 个。

不必保存全部数字的排序结果。只维护当前最大的 `k` 个：

```
[python]
import heapq

def top_k_largest(nums, k):
    heap = []

    for x in nums:
        heapq.heappush(heap, x)

        if len(heap) > k:
            heapq.heappop(heap)

    return heap
```

这里为什么使用最小堆？

> 堆中保存的是当前最大的 k 个数；堆顶是其中最小的一个，也就是最应该被淘汰的门槛。

处理每个数后：

```
heap =          k        k
```

这就是循环不变量。

7.1 复杂度

堆的大小最多为 k：

```
O(n log k)
O(k)
```

如果完整排序，则时间通常为 O(n log n)。

当 k 远小于 n 时，小堆更合适。

---

8. 第 k 大元素

维护最大的 k 个元素后，堆顶正好是它们中最小的，即整个数组第 k 大：

```
[python]
import heapq

def find_kth_largest(nums, k):
    heap = []

    for x in nums:
        heapq.heappush(heap, x)

        if len(heap) > k:
            heapq.heappop(heap)

    return heap[0]
```

也可以使用：

```
[python]
heapq.nlargest(k, nums)
```

但学习阶段应理解“大小为 k 的最小堆”为什么成立。

8.1 k 大与 k 小怎样选择堆

容易混淆时，不要死记口诀，问：

> 堆满以后，下一次最需要快速淘汰谁？

- 保留最大的 k 个：淘汰当前最小者，所以用最小堆；
- 保留最小的 k 个：淘汰当前最大者，所以用最大堆。

---

9. 高频题：前 k 个高频元素

先使用哈希表统计频率：

```
[python]
counts = {}

for x in nums:
    counts[x] = counts.get(x, 0) + 1
```

再维护频率最大的 `k` 项：

```
[python]
import heapq

def top_k_frequent(nums, k):
    counts = {}

    for x in nums:
        counts[x] = counts.get(x, 0) + 1

    heap = []

    for value, frequency in counts.items():
        heapq.heappush(heap, (frequency, value))

        if len(heap) > k:
            heapq.heappop(heap)

    return [value for frequency, value in heap]
```

如果有 `m` 个不同元素：

```
 O(n)
O(m log k)
 O(m + k)
```

返回顺序通常不重要；如果题目要求特定顺序，需要额外处理。

---

10. 高频题：合并多个有序链表

有 `k` 条有序链表。任何时刻，答案中的下一个最小节点一定来自各链表当前头节点中的最小者。

因此堆中不需要放入所有节点，只放每条链表当前可选的第一个节点。

```
[python]
import heapq

def merge_k_lists(lists):
    heap = []

    for i, node in enumerate(lists):
        if node is not None:
            heapq.heappush(heap, (node.val, i, node))

    dummy = ListNode()
    tail = dummy

    while heap:
        value, list_index, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next

        if node.next is not None:
            heapq.heappush(
                heap,
                (node.next.val, list_index, node.next),
            )

    return dummy.next
```

为什么保存 `list_index`？

当两个节点值相等时，Python 会继续比较元组第二项。整数下标可以稳定地区分它们，避免直接比较 `ListNode` 对象。

若总节点数为 `n`，堆大小最多为 `k`：

```
O(n log k)
O(k)   #
```

---

11. 堆与排序的选择

更适合排序

- 最终需要所有元素完全有序；
- 数据量不大；
- 只进行一次离线处理；
- 排序后的顺序还会被多次使用。

更适合堆

- 只需要最大或最小的少量元素；
- 数据持续到来；
- 要反复取出当前最优元素；
- `k` 远小于 `n`；
- 搜索过程中候选状态的优先级不断变化。

不要因为题目出现“最大”或“最小”就机械使用堆。只求一次最大值时，线性扫描 `O(n)` 更简单。

---

12. 常见错误

12.1 把堆当作有序数组

只能保证 `heap[0]` 是最小值，不能假设 `heap[1]` 是第二小。

12.2 直接使用 `heap.pop(0)`

这样既破坏堆操作接口，又会触发列表整体移动。应使用：

```
[python]
heapq.heappop(heap)
```

12.3 最大堆忘记取负两次

压入负数，弹出后也要恢复符号。

12.4 `k` 的含义混乱

先明确堆中保存的是“要留下的人”，还是“等待处理的候选”。写出一句不变量再编码。

12.5 元组中的对象不可比较

加入唯一序号：

```
[python]
(priority, sequence, object)
```

12.6 在空堆上读取 `heap[0]`

先确认：

```
[python]
if heap:
    smallest = heap[0]
```

12.7 修改了不该修改的输入

`heapify(nums)` 会原地改变 `nums`。题目不允许修改时先复制。

---

13. 本讲检查表

```
1.
2.
3.
4.                k
5.
6.
7.
8.
9.          heapify
10.            n    k
```

14. 本讲总结

1. 堆只维护局部顺序，堆顶是当前最优元素。
2. Python `heapq` 默认是最小堆；最大堆常用相反数模拟。
3. 插入和弹出堆顶为 `O(log n)`，查看堆顶为 `O(1)`。
4. Top K 常用大小为 `k` 的堆，把复杂度降为 `O(n log k)`。
5. 堆中可以保存 `(  ,  ,  )`。
6. 解题前先写清楚：堆里保留什么，堆顶为什么是下一步需要的元素。